DEPLOYMENT SERIES · 4 OF 6

The Deploy Workflow

Build (doc 3) happens on every push, automatically. Deploy happens on a deliberate signal only — a git tag — and that gate is the whole design.

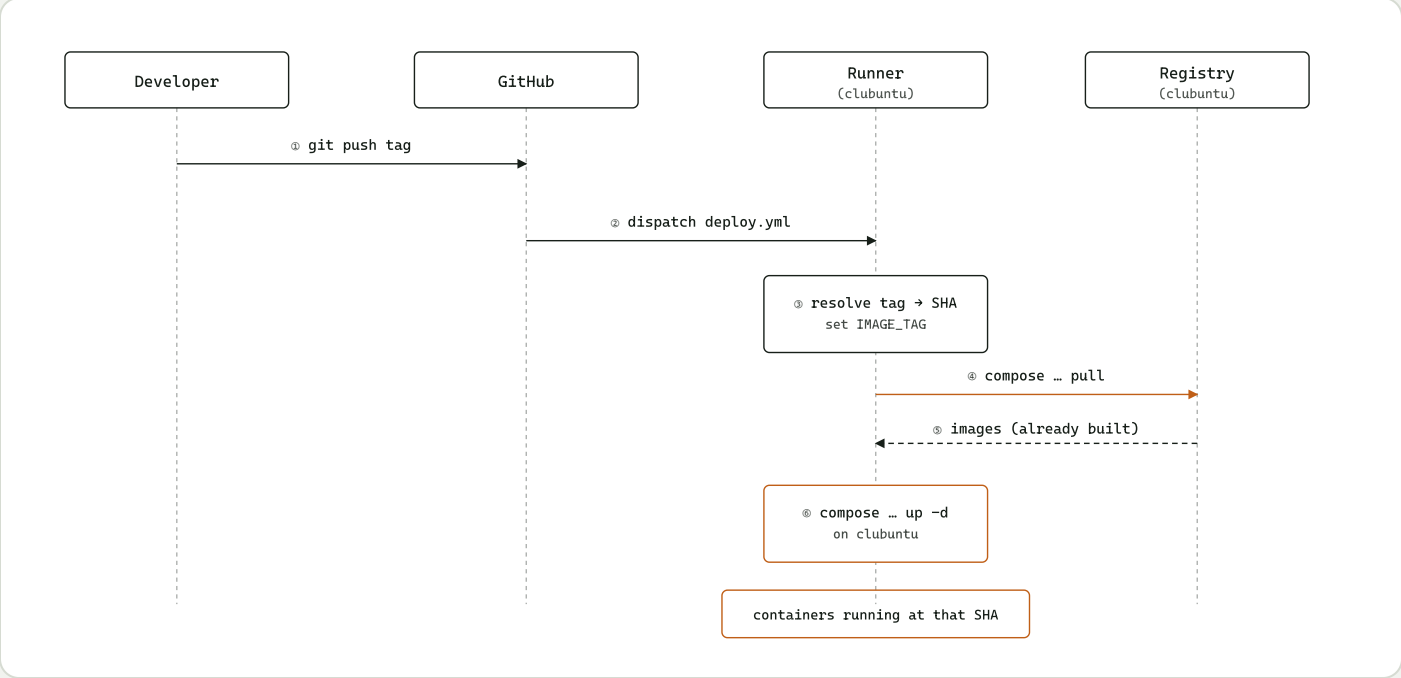
---

concept doc — no execution in this step 2026-08-16

01 Why a tag, not a push

Auto-deploying on every push to `main` means a half-finished commit can take down whatever's running. A tag is a deliberate, separate action — `git tag v1.4.0 && git push origin v1.4.0` — that only happens when a build is actually meant to go live.

02 What happens, in order



Step ④ pulls an image that doc 3 already built. Nothing here compiles anything — which is exactly why rollback (next section) costs one pull, not a rebuild.

03 Rollback is this same diagram

A rollback tag — `v1.4.0-rollback`, pointing at an older commit — runs through the identical sequence above. Step ③ resolves it to an older SHA instead of the newest one; step ④ pulls that older image, which is already sitting in the registry because it was built the same way, days or weeks ago. No special rollback code exists — it's the same six steps, given a different tag.

04 The file

```
name: deploy
on:
  push:
    tags: ['v*']

jobs:
  deploy:
    runs-on: [self-hosted, clubuntu]
    steps:
      - uses: actions/checkout@v4
      - run: |
          SHA=$(git rev-list -n 1 ${GITHUB_REF_NAME})
          echo "IMAGE_TAG=${SHA::7}" >> $GITHUB_ENV
      - run: |
          cp docker-compose.deploy.yml ~/codercommands/docker-compose.deploy.yml
      - run: |
          cd ~/codercommands
          docker compose -f docker-compose.deploy.yml pull
          docker compose -f docker-compose.deploy.yml up -d
```

The fresh checkout is disposable; `.env` lives only in the persistent deploy directory, so the compose file moves to where `.env` already is — never the other way around. `~/codercommands` is a placeholder for that real path.
